Semester	B.E. Semester VII – CMPN
Subject	R-Programming
Subject Professor In-charge	Prof. Sanjeev Dwivedi
Assisting Teachers	Prof. Sanjeev Dwivedi
Laboratory	

Student Name	Sakshi Patil	
Roll Number	23102A0046	
Grade and Subject Teacher's Signature		

Experiment Number	08
Experiment Title	Deploying a Machine Learning Model as a REST API in R using plumber
Objectives (Skill Set / Knowledge Tested / Imparted)	• To train a Random Forest classification model in R on the Iris dataset and evaluate it on a held-out test set • To save a trained model to disk (RDS format) and reload it for later use • To expose the trained model as a REST API using the plumber package • To implement a health-check endpoint and prediction endpoints using GET (query parameters) and POST (JSON body) • To test the API from R using httr / jsonlite and from the command line using curl, and verify HTTP status codes
Description	This experiment demonstrates how a machine learning model built in R can be deployed as a web service. A Random Forest classifier (100 trees) was trained on the Iris dataset using an 80/20 train-test split (seed 42) to predict the flower species (setosa, versicolor, virginica) from sepal length, sepal width, petal length and petal width. The model achieved a test accuracy of 96.67% and was saved as iris_model.rds. An API file (api.R) was then written using the plumber package. It loads the saved model and defines three endpoints: GET /health (health check), GET /predict (inputs passed as query parameters) and POST /predict (inputs passed as a JSON body). Each prediction endpoint returns the predicted species together with the class probabilities. A launcher script (run_api.R) starts the server on 127.0.0.1, port 8000, with automatic Swagger documentation at /__docs__/. The API was started in the background and tested using the httr and jsonlite packages (health check, GET and POST requests with HTTP status codes) and with curl commands. Finally, the server process was stopped.
Program	**GitHub link:** <paste your GitHub repository link here> Language / Platform: R (Google Colab R runtime) Packages: plumber, randomForest, jsonlite, httr Files created: iris_model.rds, api.R, run_api.R, api.log

Output

1. Model training and evaluation (Random Forest, 100 trees)

Test accuracy: 0.9666667 (96.67%). Model saved as iris_model.rds

Confusion matrix (rows = predicted, columns = actual):

setosa	9	0	0
versicolor	0	10	0
virginica	0	1	10

2. API started (plumber)

```
Running plumber API at http://127.0.0.1:8000
Running swagger Docs at http://127.0.0.1:8000/__docs__/
```

3. API endpoints

GET	/health	–	status, model, message
GET	/predict	Query parameters	prediction + probabilities
POST	/predict	JSON body	prediction + probabilities

4. API test results (httr / jsonlite)

GET /health	–	status = "ok", model = "RandomForest", message = "API is running"	200
GET /predict	sepal_length = 5.1, sepal_width = 3.5, petal_length = 1.4, petal_width = 0.2	prediction = setosa (setosa 1, versicolor 0, virginica 0)	200
POST /predict	sepal_length = 6.7, sepal_width = 3.0, petal_length = 5.2, petal_width = 2.3	prediction = virginica (setosa 0, versicolor 0, virginica 1)	200

Status codes: 200 200 200

The same endpoints were also called with curl (GET /predict and POST /predict with a JSON body) and the server was then stopped with pkill.

Conclusion

The experiment successfully demonstrated deploying a machine learning model as a REST API in R. A Random Forest model trained on the Iris dataset reached a test accuracy of 96.67%, with only one versicolor flower misclassified as virginica, and was saved to disk and reloaded by the API so that it does not need to be retrained for every request.

Using the plumber package, the model was exposed through a health-check endpoint and GET and POST prediction endpoints that return the predicted species with class probabilities. All three test requests returned HTTP status 200 and gave correct predictions (setosa and virginica for the two sample inputs). This shows how a trained R model can be made available to web applications and other programs, which is the basic step for putting machine learning models into

	production.